

ORBITLINK

SATELLITE INTERNET NETWORK

SQL DATABASE PROJECT REPORT

Prepared By

R Hemant Kumar Patra

Database: PROJECT_3

Technology: MySQL / SQL

A professional case-study report documenting the OrbitLink relational database, SQL analysis, data integrity, performance optimization, security, views, and 75 operational scenarios.

2026 • Academic / Portfolio Project

1. Executive Summary

OrbitLink operates a low-orbit satellite internet network serving customers in remote and underserved regions. The database connects satellite infrastructure, ground stations, customers, subscriptions, maintenance activity and individual internet usage sessions. The supplied project describes a six-table relational design and 75 real operational situations that can be solved with SQL.

Project Element	Source-supported detail
Database	project_3
Core tables	6 connected tables
Project scenarios	75 real-world SQL cases
Dataset description	20,000 records across six tables
Primary technology	MySQL / SQL
Key areas	Queries, joins, aggregation, CTEs, window functions, constraints, indexing, views and roles

2. Business Scenario

OrbitLink operates a mega-constellation of low-orbit satellites that beams broadband internet to customers in remote and underserved regions. Satellite signals are relayed through ground stations, while customer sessions capture usage, duration and latency information. These records support network engineering, billing and finance, customer support, and satellite maintenance.

Operational Questions Supported

- Identify low-activity or inactive subscriptions.
- Monitor satellite capacity, utilization and connection quality.
- Analyze customer usage and revenue by country and plan.
- Track maintenance incidents and satellite performance.
- Improve search performance through indexing.
- Protect data using constraints, roles and permissions.
- Create reusable reporting views for support and finance teams.

3. Database Architecture & Schema

The project is built around six connected tables. Primary and foreign-key relationships allow operational data to be analyzed across the satellite, customer, subscription, maintenance and usage domains.

Table	Primary Key	Purpose
Satellites	satellite_id	Satellite fleet, altitude, launch date and operational status
GroundStations	station_id	Ground-station name, country and capacity
Customers	customer_id	Customer identity, country, plan type and signup date
Subscriptions	subscription_id	Customer service subscriptions, satellite links, fees and status
MaintenanceRecords	record_id	Maintenance activity recorded against satellites
UsageLogs	log_id	Internet-session date, data usage, latency and subscription link

Relationship Flow

Customers → **Subscriptions** → **Satellites**

Subscriptions also connect to **UsageLogs**, while satellites connect to **MaintenanceRecords**. GroundStations represent the network's terrestrial relay layer.

4. SQL Techniques Demonstrated

Technique	Application in OrbitLink
Filtering & Projection	SELECT, WHERE, DISTINCT, ORDER BY and LIMIT for targeted retrieval.
Aggregation	COUNT, SUM, AVG, MIN and MAX with GROUP BY / HAVING.
Joins	INNER JOIN and LEFT JOIN for connected operational reporting and orphan detection.
Conditional Logic	CASE expressions for connection quality, utilization and generation tiers.
CTEs	WITH clauses for intermediate calculations and readable multi-stage analysis.
Window Functions	LAG, LEAD, RANK, DENSE_RANK, ROW_NUMBER and NTILE.
Date & String Functions	DATEDIFF, DATE_SUB, YEAR, QUARTER, CONCAT, TRIM, UPPER, LOWER and SUBSTRING.
Data Integrity	PRIMARY KEY, UNIQUE, NOT NULL, DEFAULT and CHECK constraints.
Performance	Composite indexing and index inspection.
Security	Roles and GRANT statements for billing and support access.
Reusable Reporting	Views for plan revenue and customer-support snapshots.

5. SQL Case Study & Query Analysis

The following section preserves the scenarios and SQL logic supplied in the OrbitLink project. Where a scenario does not contain a SQL statement in the supplied source, it is explicitly marked rather than being replaced with an invented query.

1 Case 1

Scenario text not found in the supplied SQL source.

SQL Solution: The supplied SQL source contains the scenario description but no SQL statement for this case. It is intentionally left as source-defined rather than inventing a solution.

2 Case 2

Scenario text not found in the supplied SQL source.

SQL Solution: The supplied SQL source contains the scenario description but no SQL statement for this case. It is intentionally left as source-defined rather than inventing a solution.

3 Case 3

Scenario text not found in the supplied SQL source.

SQL Solution: The supplied SQL source contains the scenario description but no SQL statement for this case. It is intentionally left as source-defined rather than inventing a solution.

4 Case 4

Scenario text not found in the supplied SQL source.

SQL Solution: The supplied SQL source contains the scenario description but no SQL statement for this case. It is intentionally left as source-defined rather than inventing a solution.

5 Case 5

a red flag for the customer success team to investigate.

SQL Solution

```
select s.*
from subscriptions s
left join usagelogs u
on s.subscription_id = u.subscription_id
where u.log_id is null;
```

Explanation: Uses a LEFT JOIN so unmatched parent records are retained, which is useful for finding missing relationships or zero-activity entities.

6 Case 6

longer exists. Track down these orphaned records.

SQL Solution

```
select u.*
```

```
from usagelogs u
left join subscriptions s
on u.subscription_id = s.subscription_id
where s.subscription_id is null;
```

Explanation: Uses a LEFT JOIN so unmatched parent records are retained, which is useful for finding missing relationships or zero-activity entities.

7 Case 7

history instantly, but this lookup has started to feel sluggish now that the log table has grown into the tens of thousands. What would you do about it?

SQL Solution

```
create index idx_usage_subscription
on usagelogs(subscription_id);
```

Explanation: Performance optimization: an index is used to make repeated filtering or lookup operations faster.

8 Case 8

adjusted by a fixed percentage. Apply this change to just that plan type's subscriptions.

SQL Solution

```
update subscriptions s
inner join customers c
on s.customer_id = c.customer_id
set s.monthly_fee = s.monthly_fee * 1.10
where c.plan_type = 'Premium';
```

Explanation: Uses relational joins to combine information from connected tables and answer a business question across the database.

9 Case 9

date, to visualize how each plan has grown.

SQL Solution

```
select c.plan_type,s.start_date,
count(*) over (partition by plan_type order by start_date) as Running_total
from subscriptions s
inner join customers c
on s.customer_id = c.customer_id;
```

Explanation: Uses relational joins to combine information from connected tables and answer a business question across the database.

10 Case 10

SQL Solution

```
select *
from satellites
where year(launch_date) =
(select year(launch_date) from satellites
where satellite_id =
(select satellite_id
from subscriptions
group by satellite_id
order by count(*) desc
limit 1));
```

Explanation: Aggregates records by the required business dimension using GROUP BY.

11 Case 11

network's heaviest users market by market.

SQL Solution

```
select c.country,c.customer_id,
sum(data_used_gb) as Total_data,
rank() over (partition by country order by sum(data_used_gb) desc) as Rnk
from customers c
inner join subscriptions s
on c.customer_id = s.customer_id
inner join usagelogs u
on s.subscription_id = u.subscription_id
group by c.country,c.customer_id;
```

Explanation: Uses RANK() with partitioning to identify the top record(s) within each group while preserving ties.

12 Case 12

satellites with zero subscribers and any subscription somehow missing a satellite link — so nothing gets left out of the audit.

SQL Solution

```
select sa.satellite_id,sa.satellite_name,s.subscription_id,s.customer_id
from satellites sa
left join subscriptions s
on sa.satellite_id = s.satellite_id
group by sa.satellite_id,sa.satellite_name,s.subscription_id,s.customer_id
order by sa.satellite_id;
```

Explanation: Uses a LEFT JOIN so unmatched parent records are retained, which is useful for finding missing relationships or zero-activity entities.

13 Case 13

it's connected to, all on a single line.

SQL Solution

```
select s.subscription_id, c.customer_name, sa.satellite_name
from Subscriptions s
join Customers c
on s.customer_id = c.customer_id
left join Satellites sa
on s.satellite_id = sa.satellite_id
group by s.subscription_id, c.customer_name, sa.satellite_name
order by s.subscription_id;
```

Explanation: Uses a LEFT JOIN so unmatched parent records are retained, which is useful for finding missing relationships or zero-activity entities.

14 Case 14

contributing less than 10% of total revenue — a pricing strategy question worth raising.

SQL Solution

```
select c.plan_type,
sum(s.monthly_fee) as total_monthly_revenue,
round(sum(s.monthly_fee) * 100.0 /(select sum(s.monthly_fee) from subscriptions),2) as revenue_percentage
from Customers c
inner join subscriptions s
on c.customer_id = s.customer_id
group by plan_type
```

```
having sum(monthly_fee) * 100.0 /(select sum(monthly_fee) from subscriptions) < 10;
```

Explanation: Uses relational joins to combine information from connected tables and answer a business question across the database.

15 Case 15

this month — these customers may be about to churn.

SQL Solution

```
select s.subscription_id,
count(u.log_id) as session_count
from Subscriptions s
left join UsageLogs u
on s.subscription_id = u.subscription_id
and year(u.session_date) = year(curdate())
and month(u.session_date) = month(curdate())
group by s.subscription_id, u.session_date
having count(u.log_id) < 3
order by s.subscription_id;
```

Explanation: Uses a LEFT JOIN so unmatched parent records are retained, which is useful for finding missing relationships or zero-activity entities.

16 Case 16

its very first maintenance record.

SQL Solution

```
select s.satellite_id, s.satellite_name,
datediff(min(m.maintenance_date), s.launch_date) as Days_passed
from Satellites s
inner join Maintenancerecords m
on s.satellite_id = m.satellite_id
group by s.satellite_id, s.satellite_name
having s.satellite_id = 30
order by s.satellite_id;
```

Explanation: Uses relational joins to combine information from connected tables and answer a business question across the database.

17 Case 17

readable label, separated by a comma.

SQL Solution

```
select customer_id,
concat(trim(customer_name), ', ', country) as Name_country
from Customers;
```

Explanation: Uses SQL string functions to clean, standardize or combine text fields for reporting.

18 Case 18

two filters are almost always used side by side. How would you set the database up so this specific combined search runs faster?

SQL Solution

```
create index idx_satellite_maintenancedate
on maintenancerecords(satellite_id, maintenance_date);
show index from maintenancerecords;
```

Explanation: Performance optimization: an index is used to make repeated filtering or lookup operations faster.

19

Case 19

ones, since this is unused capacity worth flagging.

SQL Solution

```
select sa.satellite_id, sa.satellite_name
from Satellites sa
left join Subscriptions s
on sa.satellite_id = s.satellite_id
and s.status = 'Active'
where s.subscription_id is null;
```

Explanation: Uses a LEFT JOIN so unmatched parent records are retained, which is useful for finding missing relationships or zero-activity entities.

20

Case 20

number of subscriptions it serves, their average session latency, the total downtime from maintenance, and a clear flag for any satellite whose average latency has crossed an unacceptable threshold.

SQL Solution: The supplied SQL source contains the scenario description but no SQL statement for this case. It is intentionally left as source-defined rather than inventing a solution.

21

Case 21

recorded incidents — these may need to be prioritized for early retirement.

SQL Solution

```
select satellite_id,
count(*) as incident_recorded
from MaintenanceRecords
group by satellite_id
having count(*) > 10;
```

Explanation: Uses GROUP BY for aggregation and HAVING to filter groups after the aggregate calculation.

22

Case 22

are currently linked to it relative to its typical capacity.

SQL Solution

```
with Satellite_usage as
(select sa.satellite_id, sa.satellite_name,
count(s.subscription_id) as Subscription_count
from satellites sa
left join subscriptions s
on sa.satellite_id = s.satellite_id
group by sa.satellite_id, sa.satellite_name)
select satellite_id, satellite_name, subscription_count,
case
when subscription_count > (select avg(subscription_count)*1.20
from Satellite_usage) then 'Overloaded'
when subscription_count < (select avg(subscription_count)*0.80
from satellite_usage) then 'Underutilized'
else 'Balanced'
end as Utilization_status
from Satellite_usage;
```

Explanation: Uses a Common Table Expression (CTE) to build an intermediate result that makes the final analysis easier to structure.

23

Case 23

SQL Solution

```
select c.plan_type,
count(s.subscription_id) as Active_subscriptions
from customers c
inner join subscriptions s
on c.customer_id = s.customer_id
where s.status = 'Active'
group by c.plan_type
order by Active_subscriptions desc;
```

Explanation: Uses relational joins to combine information from connected tables and answer a business question across the database.

24

Case 24

three different targeted upsell campaigns.

SQL Solution

```
with CustomerUsage as
(select c.customer_id,c.customer_name,
sum(u.data_used_gb) as Total_data
from customers c
left join subscriptions s
on c.customer_id = s.customer_id
left join usagelogs u
on s.subscription_id = u.subscription_id
group by c.customer_id,c.customer_name)
select customer_id,customer_name,total_data,
ntile(3) over (order by total_data) as Usage_tier
from CustomerUsage;
```

Explanation: Uses NTILE() to divide customers into approximately equal-sized analytical tiers.

25

Case 25

to help support agents quickly diagnose complaints.

SQL Solution

```
select log_id,subscription_id,session_date,latency_ms,
case
when latency_ms <100 then 'Excellent'
when latency_ms <300 then 'Acceptable'
else 'Poor'
end as Connection_quality
from Usagelogs;
```

Explanation: Uses CASE logic to convert numeric or categorical conditions into business-friendly labels.

26

Case 26

are generating the most revenue per customer.

SQL Solution

```
with Country_customers as
(select country,
count(customer_id) as Total_customers
from customers
group by country),
Country_revenue as
(select c.country,
sum(s.monthly_fee) as Total_revenue
from customers c
```

```

inner join subscriptions s
on c.customer_id = s.customer_id
group by c.country)
select cc.country, cc.total_customers, cr.total_revenue,
round(cr.total_revenue / cc.total_customers, 2) as Revenue_per_customer
from Country_customers cc
inner join country_revenue cr
on cc.country = cr.country
order by Revenue_per_customer desc;

```

Explanation: Uses a Common Table Expression (CTE) to build an intermediate result that makes the final analysis easier to structure.

27

Case 27

sessions have been logged across the entire network so far.

SQL Solution

```

select count(*) as Total_session
from UsageLogs;

```

Explanation: Uses aggregate functions to turn detailed records into a useful business metric.

28

Case 28

Correct this in one sweep rather than customer by customer.

SQL Solution

```

update Customers
set country = 'Egypt'
where country = 'wrongcountry';

```

Explanation: Updates existing records to reflect a business change while limiting the modification to the intended rows.

29

Case 29

lookups on the usage logs table. Check what's currently in place.

SQL Solution

```

show index from usagelogs;

```

Explanation: Performance optimization: an index is used to make repeated filtering or lookup operations faster.

30

Case 30

Explain what should happen to this record and why it must not slip through quietly.

SQL Solution: The supplied SQL source contains the scenario description but no SQL statement for this case. It is intentionally left as source-defined rather than inventing a solution.

31

Case 31

to plan network maintenance windows around quieter periods.

SQL Solution

```

select session_date,
sum(data_used_gb) as Highest_data_used,
dayname(session_date) as Day_of_week

```

```
from UsageLogs
group by session_date, data_used_gb
order by Highest_data_used desc
limit 1;
```

Explanation: Aggregates records by the required business dimension using GROUP BY.

32 Case 32

Clean these up before they go into the network status report.

SQL Solution

```
set sql_safe_updates = 0;
update groundstations
set station_name = trim(station_name);
set sql_safe_updates = 1;
```

Explanation: Uses SQL string functions to clean, standardize or combine text fields for reporting.

33 Case 33

Remove exactly those records.

SQL Solution

```
delete from usagelogs
where session_date < '2025-01-01';
```

Explanation: Deletes only the records matching the specified business condition.

34 Case 34

patterns.

SQL Solution

```
select *
from usagelogs
where month(session_date) = 3
and year(session_date) = year(curdate());
```

Explanation: Uses SQL date functions to calculate elapsed time or classify records by calendar period.

35 Case 35

the network team can understand consumption patterns at a glance.

SQL Solution

```
select log_id, subscription_id, data_used_gb,
case
when data_used_gb < 5 then 'Light'
when data_used_gb <= 20 then 'Moderate'
else 'Heavy'
end as Usage_category
from usagelogs;
```

Explanation: Uses CASE logic to convert numeric or categorical conditions into business-friendly labels.

36 Case 36

incident really is.

SQL Solution

```
select avg(downtime_minutes) as Average_downtime_minutes
from maintenancerecords;
```

Explanation: Uses aggregate functions to turn detailed records into a useful business metric.

37

Case 37

database needs to be fully withdrawn with immediate effect.

SQL Solution

```
revoke all privileges,
grant option
from 'contractor_user'@'localhost';
flush privileges;
```

Explanation: Security and access control: a role is created and permissions are granted according to the user's job responsibility.

38

Case 38

SQL Solution

```
select round(avg(orbit_altitude_km), 2) as avg_orbit_alt
from Satellites;
```

Explanation: Uses aggregate functions to turn detailed records into a useful business metric.

39

Case 39

count and total downtime. Set up something they can pull directly, without you explaining the underlying tables every time they ask.

SQL Solution

```
create view SatelliteSummary as
select sa.satellite_id, sa.satellite_name,
count(s.subscription_id) as total_subscriber,
sum(downtime_minutes) as total_downtime
from Satellites sa
inner join Subscriptions s
on sa.satellite_id = s.satellite_id
inner join MaintenanceRecords m
on s.satellite_id = m.satellite_id
group by sa.satellite_id, sa.satellite_name
order by sa.satellite_id ;
select * from SatelliteSummary;
```

Explanation: Creates a reusable database view so reporting users can access a prepared result without rebuilding the full query.

40

Case 40

the current one, to help spot a pattern building up across consecutive sessions.

SQL Solution

```
select subscription_id, session_date, data_used_gb,
lead(data_used_gb) over (partition by subscription_id order by session_date, log_id) as Next_session_data_usage
from UsageLogs;
```

Explanation: Uses the LEAD() window function to compare a row with the next record in sequence.

41

Case 41

only the current reading and the two immediately before it.

SQL Solution

```
select satellite_id, session_date, latency_ms,
avg(latency_ms) over (
partition by satellite_id order by session_date, log_id
rows between 2 preceding and current row) as Moving_average_latency
from usagelogs u
inner join subscriptions s
on u.subscription_id = s.subscription_id;
```

Explanation: Uses relational joins to combine information from connected tables and answer a business question across the database.

42

Case 42

next marketing campaign.

SQL Solution

```
select min(latency_ms) as Lowest_Latency
from UsageLogs;
```

Explanation: Uses aggregate functions to turn detailed records into a useful business metric.

43

Case 43

single maintenance record — either remarkably reliable or worryingly unmonitored.

SQL Solution

```
select s.satellite_id, s.satellite_name
from Satellites s
left join MaintenanceRecords m
on s.satellite_id = m.satellite_id
where m.record_id is null;
```

Explanation: Uses a LEFT JOIN so unmatched parent records are retained, which is useful for finding missing relationships or zero-activity entities.

44

Case 44

phases of the network's rollout.

SQL Solution

```
select customer_id, customer_name,
year(signup_date) as Year_signedup,
quarter(signup_date) as quarter_signedup
from Customers
group by customer_id, Customer_name
order by customer_id;
```

Explanation: Aggregates records by the required business dimension using GROUP BY.

45

Case 45

sure this kind of entry is rejected the moment it's submitted?

SQL Solution


```
alter table satellites
add constraint check_orbit_altitude
check (orbit_altitude_km>=0);
insert into satellites
(satellite_id, satellite_name, orbit_altitude_km, launch_date)
values
(40, 'OrbitLink-040', -638, '2024-12-14', 'Retired');
```

Explanation: Schema-level data integrity: the table definition is strengthened with a constraint so invalid values are rejected.

46

Case 46

clean, uniformly capitalized list for the billing system.

SQL Solution

```
select customer_id, customer_name,
concat(upper(left(customer_name,1)),
lower(substring(customer_name,2))) as Standardized_name
from customers;
```

Explanation: Uses SQL string functions to clean, standardize or combine text fields for reporting.

47

Case 47

who's eligible for the one-year loyalty discount.

SQL Solution

```
select customer_id, customer_name, signup_date,
datediff(curdate(), signup_date) as Days_Passed
from Customers;
```

Explanation: Uses SQL date functions to calculate elapsed time or classify records by calendar period.

48

Case 48

haven't logged any usage yet still appear on the list rather than disappearing.

SQL Solution

```
select s.subscription_id, s.customer_id,
max(u.session_date) as Most_recent_session
from subscriptions s
left join usagelogs u
on s.subscription_id = u.subscription_id
group by s.subscription_id, s.customer_id;
```

Explanation: Uses a LEFT JOIN so unmatched parent records are retained, which is useful for finding missing relationships or zero-activity entities.

49

Case 49

heaviest sessions, making sure that if two sessions used the exact same amount of data, neither one gets unfairly skipped in the ordering.

SQL Solution

```
select log_id, subscription_id, session_date, data_used_gb,
dense_rank() over (order by data_used_gb desc) as Usage_rank
from usagelogs
order by Usage_rank;
```

Explanation: Uses DENSE_RANK() to order records while giving tied values the same rank without skipping the next rank.

50

Case 50

Generation' (after 2023), to help engineers prioritize fleet upgrades.

SQL Solution

```
select satellite_id,satellite_name,launch_date,
case
when year(launch_date) < 2021 then 'Legacy'
when year(launch_date) between 2021 and 2023 then 'Mid-Generation'
else 'New Generation'
end as Generation_tier
from Satellites;
```

Explanation: Uses CASE logic to convert numeric or categorical conditions into business-friendly labels.

51

Case 51

running a large upload job worth understanding better.

SQL Solution

```
select log_id,subscription_id,
sum(data_used_gb) as Highest_data_usage
from usagelogs
group by log_id,subscription_id
order by Highest_data_usage desc
limit 1;
```

Explanation: Aggregates records by the required business dimension using GROUP BY.

52

Case 52

would permanently make this impossible going forward?

SQL Solution

```
alter table usagelogs
add constraint Check_latency
check (latency_ms >= 0);
```

Explanation: Schema-level data integrity: the table definition is strengthened with a constraint so invalid values are rejected.

53

Case 53**SQL Solution**

```
select *
from usagelogs
where subscription_id =
(select subscription_id
from usagelogs
order by data_used_gb desc
limit 1);
```

Explanation: Retrieves and transforms the required records using SQL filtering, projection and relational logic.

54

Case 54

90 milliseconds.

SQL Solution

```

with CountryLatency as
(select c.country,
avg(u.latency_ms) as avg_latency
from customers c
inner join subscriptions s
on c.customer_id = s.customer_id
inner join usagelogs u
on s.subscription_id = u.subscription_id
group by c.country)
select distinct
c.plan_type
from customers c
inner join CountryLatency cl
on c.country = cl.country
where cl.avg_latency > 90;

```

Explanation: Uses a Common Table Expression (CTE) to build an intermediate result that makes the final analysis easier to structure.

55

Case 55

performing worse than the network-wide average.

SQL Solution

```

select s.satellite_id,
avg(u.latency_ms) as Satellite_avg_latency,
(select avg(latency_ms) from usagelogs) as Network_avg_latency,
case
when avg(u.latency_ms) > (select avg(latency_ms) from usagelogs)
then 'Worse than network average'
else 'Better than network average'
end as Performance_score
from usagelogs u
inner join subscriptions s
on u.subscription_id = s.subscription_id
group by s.satellite_id;

```

Explanation: Uses relational joins to combine information from connected tables and answer a business question across the database.

56

Case 56

anywhere in the entire dataset.

SQL Solution

```

select sa.satellite_id, sa.satellite_name, u.subscription_id,
max(u.data_used_gb) as Highest_Data_used
from Satellites sa
inner join subscriptions s
on sa.satellite_id = s.satellite_id
inner join Usagelogs u
on s.subscription_id = u.subscription_id
group by sa.satellite_id, sa.satellite_name, u.subscription_id
order by Highest_Data_used desc
limit 1;

```

Explanation: Uses relational joins to combine information from connected tables and answer a business question across the database.

57

Case 57

30 characters) so it fits the map interface properly.

SQL Solution

```

select station_id, station_name,
length(station_name) as length_of_station_name
from groundstations
where length(station_name) > 30;

```

Explanation: Retrieves and transforms the required records using SQL filtering, projection and relational logic.

58

Case 58**SQL Solution**

```
select count(distinct country) as Total_countries
from Customers;
```

Explanation: Uses aggregate functions to turn detailed records into a useful business metric.

59

Case 59

the end of their expected operational life.

SQL Solution

```
select satellite_id,satellite_name,launch_date
from Satellites
where launch_date < date_sub(curdate(), interval 5 year);
```

Explanation: Uses SQL date functions to calculate elapsed time or classify records by calendar period.

60

Case 60

identify which subscriptions have gone quiet in the last 30 days.

SQL Solution

```
With RecentSessions as
(select subscription_id,
max(session_date) as Latest_session
from usagelogs
group by subscription_id)
select s.subscription_id,rs.Latest_session
from subscriptions s
left join RecentSessions rs
on s.subscription_id = rs.subscription_id
where rs.latest_session is null
or rs.latest_session < date_sub(curdate(),interval 30 day);
```

Explanation: Uses a Common Table Expression (CTE) to build an intermediate result that makes the final analysis easier to structure.

61

Case 61

the support staff. Set up an access level that reflects this difference.

SQL Solution

```
create role 'billing_supervisor';
grant update on project_3.Subscriptions
to 'billing_supervisor';
grant update (monthly_fee)
on project_3.Subscriptions
to 'billing_supervisor';
```

Explanation: Security and access control: a role is created and permissions are granted according to the user's job responsibility.

62

Case 62

generated from all active subscriptions.

SQL Solution

```
select sum(monthly_fee) as Total_monthly_Revenue
from Subscriptions
where status = 'Active';
```

Explanation: Uses aggregate functions to turn detailed records into a useful business metric.

63

Case 63

subscription data, never change any of it. Set up their access accordingly.

SQL Solution

```
create role 'support_agent';
grant select on project_3.Customers
to 'support_agent';
```

Explanation: Security and access control: a role is created and permissions are granted according to the user's job responsibility.

64

Case 64

satellite records so this detail can be captured.

SQL Solution

```
alter table Satellites
add column manufacturer varchar(100);
select Satellite_id, satellite_name, manufacturer
from Satellites;
```

Explanation: Schema evolution: the table structure is adjusted to support a new business requirement or enforce a required field.

65

Case 65

prevent that incomplete record from being saved?

SQL Solution

```
alter table Subscriptions
modify customer_id int not null;
```

Explanation: Schema evolution: the table structure is adjusted to support a new business requirement or enforce a required field.

66

Case 66

more than 20 subscribers — these may be nearing capacity.

SQL Solution

```
select sa.satellite_id, sa.satellite_name,
count(s.subscription_id) as total_subscriptions
from Satellites sa
inner join Subscriptions s
on sa.satellite_id = s.satellite_id
group by sa.satellite_id, sa.satellite_name
having count(s.subscription_id) > 20
order by sa.satellite_id;
```

Explanation: Uses relational joins to combine information from connected tables and answer a business question across the database.

67

Case 67

during a fleet update. What should stop this from happening?

SQL Solution

```
alter table Satellites
modify satellite_name varchar(30) not null unique;
```

Explanation: Schema evolution: the table structure is adjusted to support a new business requirement or enforce a required field.

68

Case 68

that satellite's other subscription rows in the process — every subscription should still show up in the result.

SQL Solution

```
select subscription_id,satellite_id,total_usage
from (select s.subscription_id,s.satellite_id,
sum(u.data_used_gb) as Total_usage,
rank() over (partition by s.satellite_id order by sum(u.data_used_gb) desc) as rnk
from subscriptions s
left join usagelogs u
on s.subscription_id = u.subscription_id
group by s.subscription_id, s.satellite_id) as ranked
where rnk = 1;
```

Explanation: Uses RANK() with partitioning to identify the top record(s) within each group while preserving ties.

69

Case 69

need additional network capacity.

SQL Solution

```
select c.country,
round(avg(u.data_used_gb),2) as Avg_data_usage
from customers c
inner join subscriptions s
on c.customer_id = s.customer_id
inner join usagelogs u
on s.subscription_id = u.subscription_id
group by c.country
having avg(u.data_used_gb) >20;
```

Explanation: Uses relational joins to combine information from connected tables and answer a business question across the database.

70

Case 70

of subscribers.

SQL Solution

```
select s.subscription_id,s.satellite_id,s.customer_id
from subscriptions s
where s.satellite_id =
(select satellite_id
from subscriptions
group by satellite_id
order by count(*) desc
limit 1);
```

Explanation: Aggregates records by the required business dimension using GROUP BY.

71 Case 71

reflects each plan type's current total monthly revenue, without re-deriving it manually every cycle.

SQL Solution

```
create view PlanTypeRevenue as
select plan_type,
sum(monthly_fee) as Total_monthly_revenue
from subscriptions s
inner join customers c
on s.customer_id = c.customer_id
group by plan_type;
select * from PlanTypeRevenue;
```

Explanation: Creates a reusable database view so reporting users can access a prepared result without rebuilding the full query.

72 Case 72

midnight and 5 AM.

SQL Solution

```
select u.*
from usagelogs u
inner join subscriptions s
on u.subscription_id = s.subscription_id
where s.customer_id = 101
and time(u.session_date) between '00:00:00' and '05:00:00';
```

Explanation: Uses relational joins to combine information from connected tables and answer a business question across the database.

73 Case 73

station's coverage footprint (matched via country).

SQL Solution: The supplied SQL source contains the scenario description but no SQL statement for this case. It is intentionally left as source-defined rather than inventing a solution.

74 Case 74

that same subscription's own typical usage — a possible sign of a compromised account or a new use case.

SQL Solution

```
select subscription_id, session_date, data_used_gb,
avg(data_used_gb) over (partition by subscription_id) as Typical_usage,
case
when data_used_gb > avg(data_used_gb) over (partition by subscription_id)
then 'Higher than typical'
else 'Normal'
end as usage_flag
from usagelogs;
```

Explanation: Uses CASE logic to convert numeric or categorical conditions into business-friendly labels.

75 Case 75

name, plan type, assigned satellite, and most recent session, without agents having to reconstruct the full picture from scratch on every call.

SQL Solution

```
create view CustomerSupportSnapshot as
with LatestUsage as
(select u.subscription_id,u.session_date,
row_number() over (partition by u.subscription_id order by u.session_date desc, u.log_id desc) as rnk
from UsageLogs u)
select c.customer_name,c.plan_type,sa.satellite_name,lu.session_date AS Most_recent_session
from Customers c
inner join Subscriptions s
on c.customer_id = s.customer_id
left join Satellites sa
on s.satellite_id = sa.satellite_id
left join LatestUsage lu
on s.subscription_id = lu.subscription_id
and lu.rnk = 1;
select * from CustomerSupportSnapshot;
```

Explanation: Creates a reusable database view so reporting users can access a prepared result without rebuilding the full query.

6. Data Integrity, Performance & Security

Data Integrity

The project uses primary keys, unique values, NOT NULL, DEFAULT and CHECK constraints to prevent invalid or incomplete records.

Performance Optimization

Composite indexing is demonstrated for satellite + maintenance-date filtering, with index inspection through SHOW INDEX.

Access Control

Separate roles are demonstrated for billing supervisors and support agents, with permissions aligned to job responsibilities.

Schema Evolution

ALTER TABLE statements show how the database can evolve when new business requirements appear, such as recording satellite manufacturers.

Reusable Reporting

Views provide stable reporting interfaces for plan revenue and customer-support snapshots.

7. Project Workflow

1. Business Scenario



2. Requirement Analysis



3. Entity Identification



4. Database Design



5. Table Creation



6. Constraints



7. Data Insertion



8. SQL Analysis



9. Data Validation



10. Performance Optimization



11. Reporting



12. Business Insights

8. Skills Demonstrated

Database

MySQL • Relational Database Design • EER / Schema Understanding

SQL

SELECT • JOIN • GROUP BY • HAVING • CASE • Subqueries • CTEs • Window Functions • Date Functions • String Functions

Data Management

Data Cleaning • Data Validation • Data Integrity

Optimization

Indexing • Query Optimization • Views

Administration

Roles • GRANT / Permissions • Controlled Access

9. Conclusion

The OrbitLink project demonstrates how SQL can be applied to a realistic satellite-internet operation rather than being limited to simple database exercises. The six-table design supports cross-functional analysis across customers, subscriptions, satellites, maintenance and usage. The 75 scenarios cover operational monitoring, customer analytics, revenue analysis, data quality, performance tuning and access control.

For an SQL/Data Analyst portfolio, the project demonstrates practical use of joins, aggregation, subqueries, CTEs, window functions, date and string operations, constraints, indexes, views and database security concepts.

10. Future Enhancement Recommendations

- Connect the SQL database to Power BI or Tableau for operational dashboards.
- Add automated alerts for degraded satellites, unusual usage and low-activity subscriptions.
- Create scheduled reporting for finance, customer support and network engineering.
- Introduce stored procedures for frequently repeated operational tasks.
- Build a data warehouse layer for long-term usage and revenue analytics.
- Add Python-based analytics or machine-learning models for anomaly detection and churn prediction.

Prepared for: R Hemant Kumar Patra

Project: OrbitLink Satellite Internet Network Database

Database: project_3